CBCS Timetable Engine

Product Requirements Document

`Capacity-Buffered Constraint Scheduling`

This document specifies the CBCS timetable generation system end to end: the capacity model and its formulas, the data model, the solver pipeline, every constraint, the disruption-handling logic, the complete edge-case register, and the honest verification status of each component as of the current build.

Document	Product Requirements Document (PRD)
System	CBCS — Capacity-Buffered Constraint Scheduling
Version	1.0
Status	Current — reflects the verified build
Audience	Engineering team; Director of Technology
Classification	Confidential — Internal

> **How to read this document**
>
> Every capability in this PRD carries an explicit verification status. Where a feature is specified but not built, or built but only proven on a non-production code path, this document says so in the same sentence as the specification. Claims marked **VERIFIED** are backed by an executed run against the real dataset (5 sections, 9 subjects, 2,265 sessions); claims marked **UNVERIFIED** are not. This convention is deliberate: an unqualified specification would misrepresent the system's readiness.

Source reconciliation. This PRD consolidates three prior artefacts that do not fully agree with one another: the original capacity-model derivation (formulas and worked examples), the Semester Simulator system-logic reference (16-part design specification), and the locked business-decision set (final scope determinations). Where they conflict, the locked decisions govern, and the divergence is documented in Section 2 rather than silently resolved.

Contents

1. Product definition and scope

1.1 Purpose

A university programme has a fixed quantity of teaching time and a curriculum that must fit inside it. Determining whether it fits — and producing a legal timetable when it does — is done manually in most institutions, by spreadsheet, and the failure modes are invisible until the term is underway. CBCS exists to make that determination computable, auditable and honest.

1.2 The core model

> **The single invariant**
>
> Teaching time is a fixed budget of slots. Every subject, practice session, activity, exam, holiday and disruption spends from the same pool. A class placement is legal only when the section ledger and the instructor ledger are simultaneously free at that time.

Two ledgers govern every placement:

- **The section ledger.** A section is a cohort of students. It can attend at most one session per time slot.
- **The instructor ledger.** An instructor exists on a single real-world timeline. A slot booked for one subject, one section or one batch blocks that time for every other. This ledger is shared across all batches and all subjects that instructor teaches — it is not scoped to a section.

A third ledger — rooms — was considered and deliberately excluded from scope (Section 1.4).

1.3 In scope

Capability	Summary
Calendar and slot budget	Semester dates, daily timings, multiple breaks with real clock times, day-state policy, holidays, exam blocks, planned activities — resolved into an exact per-section slot budget.
Academic structure	Batch → branch → section → subject hierarchy with multi-section subject assignment, credits, and exact session counts.
Automatic faculty assignment	The system assigns instructors. Operators supply faculty data and capability; they do not hand-pick an instructor per delivery.
Timetable generation	Constraint-satisfaction solver with an exact fallback, a sound infeasibility prover, week-aligned chunking, and independent post-generation validation.
Disruption handling	Faculty absence (with leave-and-return substitution), sudden activities, dated holidays — all through one unified cascade.
Versioning and approval	Per-batch immutable versions with effective-date ranges, a permission-driven publishing workflow, and an audit trail.
Roles and access	Super Admin, Admin, BOA and a configurable Approver/Publisher, enforced server-side.
Import and export	Template-driven bulk import with mandatory preview; PDF, Excel, CSV and JSON export with identical session identities across formats.

1.4 Out of scope, with rationale

These exclusions are deliberate decisions, not omissions. Each was considered during design and consciously removed.

Excluded	Rationale
Rooms as a scheduled resource	Room allocation is determined by the university and handed to the programme; the timetable does not own that resource. Modelling it would introduce a third ledger and a class of conflict the system cannot actually resolve. Rooms are never mandatory and never block generation.
Exam scheduling	Exam periods are treated as calendar blocks only. The system reserves the dates and removes their slots from the teaching budget. It does not produce exam timetables, seating plans, or invigilation rosters — those are owned elsewhere.

Excluded	Rationale
Invigilator assignment	Follows from the above. No invigilator availability check, and no rule preventing an instructor invigilating their own subject.
Session content and delivery	The content team owns session plans and sequence; the instructor team owns completion marking, recordings and analysis. CBCS consumes a session count and sequence per subject and, optionally, a completion feed. It does not store content or render it to students.
Production operations	Authentication (SSO), deployment, monitoring, concurrency control, backups and the production database are outside this scope and are the reason the complete product is not production-ready.

2. Scope reconciliation — designed / locked / built

Three source documents preceded this PRD and they do not agree. The system-logic reference describes a broader design than the locked business decisions permit, and the current build is narrower still. This section reconciles all three explicitly. It is the most operationally important table in this document: it distinguishes "we decided not to" from "we have not yet".

*Legend: **Designed** = specified in the system-logic reference. **Locked** = the final business decision. **Built** = verified state of the current code on the production path.*

Feature	Designed	Locked	Built (production path)
Slot budget from calendar	Yes	Yes	VERIFIED — 686 slots/section
Multiple breaks at real clock times	Yes	Yes	VERIFIED — periods fitted around breaks
Generalised day-state model (any weekday: full/half/off, with cadence)	Yes — explicitly: "Saturday was never actually special"	Not restated	NOT BUILT — only 7 fixed Saturday combinations. "Alternate Wednesday off" is inexpressible. See 12.1
Holidays — multi-date and ranges	Yes	Yes	BUILT — breadth cases untested
Date-specific day override (force a working day off, or an off day working)	Yes — required by the recovery mechanism	Yes	UNVERIFIED — not exercised
Exams as calendar blocks	Yes	Yes — block only	VERIFIED — 217 slots removed pre-chunk
Exam scheduling / seating / invigilators	Yes — with clash and gap rules	NO — out of scope	Correctly absent
Rooms as a third ledger	Yes — typed, capacity-checked	NO — out of scope	Correctly absent
Automatic faculty assignment	Yes	Yes — core purpose	VERIFIED
Instructor weekly cap	Yes — daily/weekly limit	Yes	VERIFIED — enforced per calendar week
Instructor daily / consecutive caps	Yes	Yes	NOT BUILT — dead columns dropped in migration 003
One owner per subject-section for the term	Yes	Yes	VERIFIED
Leave-and-return substitution (owner resumes after absence)	Yes — explicitly not a handover	Yes	VERIFIED — per-session substitute; owner unchanged
Full absence taxonomy (9 distinct cases)	Yes	Yes	PARTIAL — see 8.1
Faculty transfers (incumbent / cover / successor)	Yes	Yes — 7 warning types	NOT AVAILABLE on the real term — refused by the chunked solver. Correct one-shot. See 12.2
Joint / concurrent sessions	Yes	Yes — optional, rare	NOT AVAILABLE on the real term — refused by the chunked solver. See 12.2
Guest sessions from other universities (we-provide vs they-provide slots)	Yes	Not restated	NOT BUILT

Feature	Designed	Locked	Built (production path)
Visiting / part-time faculty availability windows	Yes	Yes	PARTIAL — availability rules threaded and honoured; no dedicated window model
Content layer — tentative/confirmed session pointer	Yes	Out of scope (content team owns)	Correctly absent
Regulatory minimum session floor	Yes	Not restated	NOT BUILT — see 12.3
Partial-week policy (skip-and-recover)	Yes	Not restated	Implicitly satisfied — exact counts are placed wherever capacity exists
Soft preferences (7 named)	Yes — backlog	Yes — non-guaranteed	PARTIAL — soft pins honoured and now reported; the other 6 not implemented
Shared elective clash detection	Yes — backlog	Not restated	NOT BUILT
Draft vs published, change log, notifications	Yes — backlog	Yes	PARTIAL — versioning and immutability verified; notification events not built

> **The most significant divergence**
>
> The system-logic reference states plainly that **any** day of the week may be set to fully working, fully off, or half-day, and that each state may carry a cadence (every week, or alternating 1st/3rd, 2nd/4th, or custom). Saturday is described as "the most common real-world example, not a special case in the logic." The current build implements seven hardcoded Saturday combinations and treats every other weekday as fixed. A rule such as "every alternate Wednesday is a half-day" cannot be expressed. This is a design regression, not a scope decision — it was never consciously excluded.

3. The capacity model

This is the foundation every other layer depends on. It converts a calendar into an integer count of teaching slots per section, then measures the curriculum against it. The result is either a positive **buffer** or a negative **deficit**; the deficit case is the single most valuable output the system produces, because it tells operations that a term is impossible before anyone attempts it.

3.1 Layer 1 — Day counting

Working days are derived by removing non-working days from the raw calendar span in stages.

```
Working Days = Total Days
- Sundays
- Saturday-policy days
- Other declared holidays
```

Worked example (the original derivation, July + August):

```
July (31) + August (31) = 62 days
- Sundays (4 in July, 5 in August) = -9
----
Working spine = 53 days
- Declared holidays (Aug 26, 28) = -2
```

Saturday policy	From the 53-day spine	Less holidays	Working days
All Saturdays holiday	53 - 9	-2	42
2nd Saturday holiday	53 - 2	-2	49
2nd and 4th Saturday holiday	53 - 4	-2	47
All Saturdays working	53 - 0	-2	51

Implementation note. *The production engine does not apply this as a closed-form subtraction. It walks every date in the semester, classifies it against that weekday's rule, and counts. The formula above is the specification; the walk is the implementation, and it is what makes date-specific overrides possible.*

3.2 Layer 2 — Slot conversion

Days become slots via the daily time budget. Breaks are removed as real clock intervals and periods resume after them; the engine does not subtract break minutes from a daily total, because that would silently create periods that overlap a break.

```
Full day : (390 min window - 40 min lunch) / 50 min = 7 slots
[09:30-16:00 = 390 min]

Half day : (390 / 2) / 50 = 3.9 -> floor = 3 slots
[no lunch break on a half day]
```

> **Two rounding decisions, made explicit**
>
> **(1)** The half-day slot count is a **floor** of 3.9, discarding approximately 0.9 of a slot per half day. This is deliberate: a partially-available period is not a teachable period. **(2)** Declared holidays are assumed not to fall on a day already removed as a Sunday or a policy Saturday. In production, holidays must be de-duplicated against the day-state walk or they will be double-counted.

3.3 Layer 3 — The master formula

Every Saturday scenario collapses into one expression. The bracket yields full working days at the full rate; the trailing term re-adds half days at the reduced rate.

```
Total Available = [ (Total Days - Sundays - Other Holidays
- Full-off Saturdays - Half Saturdays) x FullDaySlots ]
+ ( Half Saturdays x HalfDaySlots )
```

The insight this encodes: **a half-working day is not a day removed.** It is a day that contributes fewer slots, so it must be pulled out of the full-day pool and re-added at the reduced rate.

3.4 The seven Saturday scenarios

The base of pure full weekdays is 42 in every scenario; working Saturdays are added on top at either the full rate (7) or the half rate (3).

Scenario	Computation	Available slots
All Saturdays holiday	42 x 7	294
2nd Sat holiday — remainder half-days	(42 x 7) + (7 x 3)	315
2nd Sat holiday — remainder full days	49 x 7	343
2nd and 4th Sat holiday — remainder half	(42 x 7) + (5 x 3)	309
2nd and 4th Sat holiday — remainder full	47 x 7	329
All Saturdays half-days	(42 x 7) + (9 x 3)	321
All Saturdays full working	51 x 7	357

3.5 Layer 4 — Demand

The curriculum team supplies a required session count per subject, derived from content duration. This number is **fixed** — content length does not care whether Saturday is a holiday — so demand is constant across every scenario while availability moves.

```
Demand_S = Required_S x Sections_S
Total Demand = Sum over all subjects S of Demand_S
```

Practice sessions are included inside Required_S, because the same instructor delivers them.

3.6 Layer 5 — Buffer

```
Buffer = Total Available - Sum( Required slots of all subjects )
```

Because required is constant while available moves, **the buffer changes only because availability changes.** That is the entire relationship, and it is why the Saturday policy matters: it is the knob that sizes the disruption reserve.

Scenario	Available	Required	Buffer
All Saturdays holiday	294	270	24
2nd and 4th Sat holiday — half	309	270	39
2nd Sat holiday — half	315	270	45
All Saturdays half	321	270	51
2nd and 4th Sat holiday — full	329	270	59
2nd Sat holiday — full	343	270	73
All Saturdays full working	357	270	87

Illustrative required figures (A=90, B=70, C=65, D=45; total 270) from the original derivation. A college expecting frequent disruption should select a working-Saturday policy precisely to carry a larger reserve — that is what these seven scenarios were computed to decide.

> **The negative case is the important one**
>
> If **Required exceeds Available**, the buffer is negative. The system must not display a negative buffer — it must refuse. A negative buffer means the syllabus does not fit in that configuration, and the correct output is a blocking error naming the exact shortfall (for example: *needs 720, has 686, short by 34*), not a minus sign. This is structurally identical to the solver's infeasibility proof, one layer earlier.

3.7 Capacity is per section, never global

Multiple sections run in parallel, so a single college-wide slot figure is misleading. Every capacity figure in the product is per section. The interface must present both a proportional view and exact integers — a donut chart alone hides the numbers that matter.

The full breakdown the interface must expose, per section:

```
Generated calendar periods
- Holidays and non-working days
- Break periods
- Exam-blocked periods
- Planned-activity periods
= Schedulable teaching slots
- Required subject sessions
= Buffer (if >= 0) or Deficit (if < 0)
```

Also surfaced: subject demand per section, total demand across sections, faculty capacity, the **most constrained section**, and the **most constrained instructor**.

3.8 The production dataset

Quantity	Value	Status
Teaching days	98	VERIFIED
Calendar weeks	19 (18 solver chunks)	VERIFIED
Slots removed pre-chunk (holidays, exams, activities)	217	VERIFIED
Teaching slots per section	686	VERIFIED
Required sessions per section (entered)	453	Input
Scheduled per section	453 — exact	VERIFIED
Under-delivered	0	VERIFIED
Over-delivered	0	VERIFIED
Free buffer per section	233	VERIFIED

Quantity	Value	Status
Total sessions across 5 sections	2,265	VERIFIED

4. Data model

4.1 Hierarchy

```
University
+- Academic year / Semester
+- Batch (e.g. 2022-2026)
+- Branch (e.g. CSE)
+- Section (e.g. CSE-A)
+- Subjects
```

The batch layer is load-bearing: timetable versions are maintained **per batch**, not per university, so two batches may hold different active versions simultaneously.

4.2 Entity fields

Entity	Fields	Notes
Batch	name, start year, end year, current academic year, current semester, program, branches, sections	Distinguishes "batch" from "year" and "semester" explicitly.
Section	code, branch, batch, enrolled subjects	The section ledger unit. One session per slot, maximum.
Subject	code, name, type, credits, required session count, manual override + reason, duration per session, max sessions per day, preferred sessions per week, lecture/practice relationship, consecutive-period requirement, topic-sequence flag, shared-subject flag, joint-session flag	Several fields are recorded but not enforced — see 4.3.
Instructor	id, name, university, department, qualified subjects, max weekly slots, availability rules, planned leave, emergency absence, current university, transfer effective date, temporary cover, successor, successor start date	The instructor ledger unit. Weekly cap is enforced per calendar week.
Session	section, subject, topic/sequence number, type (lecture/practice), owner instructor, substitute instructor, day index, period index, date, fixed flag, joint flag	The substitute field is what makes leave-and-return expressible without changing ownership.
Fixed session (pin)	scope, date, period, subject or activity, instructor, reason, hard-lock or soft-preference	Hard pins are honoured absolutely; soft pins are honoured where possible and reported when not.
Timetable version	university, batch, version number, name, status, created by, approved by, created timestamp, effective from, effective until, reason for change, previous version, sessions changed, validation result, solver result, published timestamp	Immutable once published.

4.3 Field enforcement status

The following table is the outcome of a field-flow trace conducted against the production solver. Every input field was traced from the interface, through persistence, into the request payload, to the solver, and back to the validator. The distinction between a field that flows and a field that **binds** is the single most consequential thing in this section.

Field	Reaches solver?	Enforced?	Validated?	Status
required_slots	Y	Y	SESSION_MISSING / SESSION_EXTRA	BINDING
max_weekly	Y	Y — per calendar week	INSTRUCTOR_OVER_WEEKLY	BINDING
max_per_day (global)	Y	Y	DAY_CAP	BINDING
has_practice / pairing	Y	Y	PRACTICE_ORDER, PRACTICE_SAME_DAY	BINDING

Field	Reaches solver?	Enforced?	Validated?	Status
topic sequence	Y	Y	SEQUENCE	BINDING
availability rules	Y	Y	AVAILABILITY	BINDING
planned absence	Y	Y	ABSENCE	BINDING
substitutions	Y	Y	qualification + own weekly cap	BINDING
pins (hard)	Y	Y	PIN_MOVED	BINDING
pins (soft)	Y	Best-effort	reported in diagnostics	BINDING (advisory)
holidays, exam blocks, activity blocks	Y — baked into the calendar pre-chunk	Y	calendar	BINDING
seed, node budgets	Y	Y	—	BINDING
max_daily (per instructor)	N	N	N	REMOVED — migration 003
max_consecutive	N	N	N	REMOVED — migration 003
preferred sessions per week	N	N	N	REMOVED — migration 003
per-subject max_per_day	N	N	N	REMOVED — global cap applies. See 12.4
consecutive-period requirement	N	N	N	REMOVED — migration 003
subject type (Theory/Lab/...)	Y	N	N	DISPLAY-ONLY — labelled. Scheduling behaviour is driven by the lecture+practice flag
credits	Seeds the slot calculation	N	N	DISPLAY-ONLY — labelled
transfers	N — refused on the chunked path	N	N	NOT AVAILABLE. See 12.2
joint sessions	N — refused on the chunked path	N	N	NOT AVAILABLE. See 12.2

> **Design rule established by this trace**
>
> A field may not exist in the interface unless it flows to the engine and its effect is observable in the output. A field that is collected but never enforced is worse than an absent feature, because the operator believes a constraint is active when it is not. Every field above is therefore **binding**, **removed**, or **explicitly labelled** as non-binding on screen. There is no fourth state.

4.4 Persistence

SQLite behind a repository abstraction, with migration-ready schema. Stable IDs, created/updated timestamps, created-by attribution, and soft deletion where appropriate. This is pilot persistence and is not a claim of production database readiness.

Migrations applied: `003_drop_dead_columns` (removed six collected-but-unenforced columns); `004_recovery_dedup_live_only` (the recovery de-duplication index formerly counted soft-deleted rows, which meant a deleted recovery item could never be re-listed — a genuinely missing session stayed missing while the gap-closing routine reported it fixed).

5. Generation pipeline

5.1 Stages

#	Stage	What happens	Can terminate here?
1	Input validation	Structural validation of the request: unknown references, malformed values, contradictory pins, unqualified transfer successors.	Yes — INVALID_INPUT
2	Necessary feasibility	Arithmetic ceilings: does demand exceed the capacity that physically exists?	Yes — PROVEN_INFEASIBLE
3	Instructor allocation	One qualified owner assigned per (section, subject), least-loaded first, balancing weekly caps.	Yes — PROVEN_INFEASIBLE
4	Structural infeasibility proof	Four sound checks run against the allocated owner map, before any search. Returns in approximately zero milliseconds with a stated contradiction.	Yes — PROVEN_INFEASIBLE
5	Search	Primary CSP solver; exact fallback if required. Bounded by a deterministic node budget.	Yes — all five statuses
6	Independent validation	The assembled grid is re-checked by a validator that does not share solver state.	Yes — REJECTED_BY_VALIDATION

5.2 Result statuses

The interface must represent each of these distinctly. A generic "generation failed" is a specification violation.

Status	Meaning	Operator action
SOLVED_BY_PRIMARY	The CSP solver produced a valid grid.	None — review and publish.
SOLVED_BY_FALLBACK	The primary solver could not settle it; the exact solver did.	None — the grid is equally valid.
PROVEN_INFEASIBLE	A mathematical contradiction exists. The configuration cannot be scheduled by any solver, ever. The specific contradiction is stated.	Change the configuration. Do not retry.
INVALID_INPUT	The request is malformed or self-contradictory.	Correct the named field.
UNRESOLVED_TIMEOUT	The node budget was exhausted. **This means "not proven either way" — never "impossible".**	Retry with a larger budget, or simplify.
REJECTED_BY_VALIDATION	The solver returned a grid and the independent validator rejected it. The violated code is named.	Escalate — this indicates a solver defect.

5.3 The structural infeasibility prover

This runs after allocation and before search. It is the highest-value component in the engine: it converts an unbounded search into an instant, explained refusal. All four checks are **sound** — they only ever prove genuine impossibility and never reject a solvable configuration.

#	Check	Reasoning
1	Instructor pairing ceiling	A lecture and its practice must occupy two slots on the same day. An instructor with p free periods on a day can therefore host at most floor(p/2) pairs that day. Summed across the term, this bounds what that instructor can deliver.
2	Per (section, subject) capacity	For a given owner, the effective placeable capacity across all days must be at least the subject’s required count for that section.
3	Section total-pairs	A section’s own days bound the total number of pairs it can receive, independent of who teaches them.
4	Per-day forced load	Where sessions are forced onto specific days, the day’s period count bounds them.

Example output. A configuration requiring 10 sessions of a subject whose owner can host at most 3 days x 2 pairs returns, in 0 ms, with zero search nodes:

```
PROVEN_INFEASIBLE
"section S1 needs 10 sessions of SUB0 but at most 3 x 2 = 6
are placeable given the owner's available days"
```

Why this matters commercially

Without a sound prover, an impossible configuration is indistinguishable from a hard one: the solver searches, exhausts its budget, and returns a timeout. The operator retries with a bigger budget, waits longer, and gets the same answer — with no explanation. The prover converts that into an instant sentence naming the exact contradiction. In testing it resolves the large majority of impossible configurations before any search begins.

5.4 Week-aligned chunking

The full term cannot be solved in one pass. Attempting the production dataset single-shot exhausts the node budget after approximately 246 seconds with zero sessions placed. The engine therefore decomposes the term into chunks.

The chunk boundary is the constraint boundary. One ISO calendar week per chunk. This is not an arbitrary size — it is what makes the instructor weekly cap enforceable, because the engine's existing per-chunk budget *becomes* a weekly cap when the chunk is exactly one week. No solver surgery was required.

Property	Value	Status
Chunk size	One ISO calendar week	VERIFIED
Chunks in production term	18	VERIFIED
Mid-week boundaries	0 — by construction	VERIFIED
Chunks solved by primary	13 of 18	VERIFIED
Session-count split across chunks	Reconciled: sum of per-chunk allocations equals the entered term total, per subject, per section. No rounding loss.	VERIFIED
Low-frequency subject distribution	Evenly spaced across the term, not front-loaded	VERIFIED — fixed; see 13.2
Topic sequence across boundaries	Monotonic across all 17 boundaries; zero pairs split	VERIFIED
Validation scope	The assembled term grid, not per chunk	VERIFIED
Determinism	Identical grid hash and per-chunk node counts over 3 runs	VERIFIED

The chunking hazard, and why validation is term-wide

If each chunk were validated in isolation, four internally valid weeks could assemble into an invalid term. The clearest example: an instructor at their weekly cap in both halves of a week that straddles a boundary. Each chunk would pass; the real week would be double. Two defences apply: chunks are week-aligned so no boundary falls mid-week, and the independent validator runs on the **assembled term grid**. Both were verified with a crafted case.

5.5 Independent validation

The validator does not share the solver's state. It reads the assembled grid and re-derives every invariant from scratch. A grid that fails is **rejected, not displayed**. This was verified by forcing a double-booking into a valid grid and confirming rejection.

Code	Invariant
INSTRUCTOR_DOUBLE_BOOKED	No instructor occupies two slots at the same time, across all batches and subjects.
SECTION_DOUBLE_BOOKED	No section attends two sessions at the same time.
INSTRUCTOR_OVER_WEEKLY	No instructor exceeds their entered weekly cap in any calendar week.
INSTRUCTOR_UNQUALIFIED	No instructor teaches a subject they are not qualified for.
SESSION_MISSING / SESSION_EXTRA	Scheduled count equals required count, exactly, per (section, subject).
SEQUENCE	Topic n is scheduled before topic n+1, for every subject and section.
PRACTICE_ORDER	A practice session never precedes its lecture.
PRACTICE_SAME_DAY	A practice session shares a day with its lecture.
DAY_CAP	No subject exceeds the per-day session cap for a section.

Code	Invariant
AVAILABILITY	No session is placed in a slot the instructor declared unavailable.
ABSENCE	No session is placed for an instructor during a recorded absence.
PIN_MOVED	A hard-pinned session occupies exactly its pinned slot.
TRANSFER_CONTINUITY	Ownership changes align with the declared transfer date.

6. Constraint specification

6.1 Hard constraints — never violated

A grid violating any of these is rejected by the independent validator and never reaches the operator.

Constraint	Statement	Verified
Section exclusivity	A section attends at most one session per slot.	Y
Instructor exclusivity	An instructor teaches at most one session per slot, across every batch, section and subject they hold.	Y
Qualification	An instructor only teaches subjects they are qualified for. Applies equally to substitutes.	Y
Weekly capacity	An instructor never exceeds their entered weekly cap in any calendar week.	Y
Exact delivery	Scheduled sessions equal required sessions, per (section, subject). No under-delivery, no surplus.	Y
Topic sequence	Topic n precedes topic n+1 for every subject and section, including across chunk boundaries.	Y
Lecture before practice	A practice session never precedes its lecture and shares its day.	Y
Availability	No session is placed where the instructor is unavailable or absent.	Y
Hard pins	A hard-locked session occupies exactly its pinned slot, including at a chunk boundary.	Y
Calendar restrictions	No session is placed on a holiday, in a break, or inside an exam or activity block.	Y
Per-day subject cap	A subject does not exceed the configured maximum sessions per day for a section.	Y

6.2 Soft preferences — best-effort, never guaranteed

The interface must state that soft preferences are not guaranteed. Where one is dishonoured, the system reports it rather than failing silently.

Preference	Status
Soft pins (use this slot if possible)	BUILT — honoured where possible; dishonour is reported in diagnostics and surfaced as a warning
Spread a subject across the week	PARTIAL — spreading is applied at placement
Avoid first period	NOT BUILT
Avoid last period	NOT BUILT
Prefer morning sessions	NOT BUILT
Limit instructor gaps	NOT BUILT
Balance instructor workload	PARTIAL — allocation is least-loaded-first, which balances by construction; there is no explicit balancing objective
Minimise changes after publication	BUILT via the cascade — minimum disruption is the default and whole-term reschedule is opt-in

6.3 Placement ordering

The solver places in decreasing order of constraint, while the grid is still empty and flexible. Tight items placed late produce dead ends.

```
1. Externally fixed slots (hard pins, external sessions)
2. Other pinned activities
3. Linked lecture+practice pairs (placed as an atomic unit)
4. Subjects with fewest qualified instructors
5. High-frequency core subjects
6. Low-frequency electives
7. Buffer and co-curricular
```

6.4 Deterministic tie-breaking

When two sections compete for one instructor, the order is fixed: **(1)** batch seniority, **(2)** subject with fewest qualified instructors, **(3)** lower section index. Determinism means identical inputs always produce an identical timetable, so any decision can be explained and reproduced. Verified over three runs with identical grid hash and node counts.

7. Instructor allocation and the shared ledger

7.1 Automatic assignment is the product

Operators supply faculty data — who exists, what they can teach, their capacity, their availability, their university. The **engine** decides who teaches what. A BOA is never asked to hand-pick an instructor per subject; that is the manual labour the system exists to remove. Only Admin or Super Admin may override an assignment, and every override requires a reason, passes validation, creates a new version, and enters the audit history.

Assignment considers: subjects the instructor can teach, assigned university, weekly capacity, availability, planned leave and absence, existing commitments, section conflicts, transfers, temporary cover, permanent successor, and workload balance.

7.2 Allocation formulas

```
Demand_S = Required_S x Sections_S

Cap = Working days x Slots per day
(one instructor occupies one timeline)

Instructors_S = ceil( Demand_S / Cap )

Global check : Sum(Demand_S) <= Total instructors x Cap

Sections/head_S = floor( Cap / Required_S )
Coverable_S = Qualified_S x Sections per head_S
Unallocated_S = max(0, Sections_S - Coverable_S)
Extra needed_S = ceil( Unallocated_S / Sections per head_S )
```

Worked example. Two months, all Saturdays working: Cap = 51 x 7 = 357. A batch with 2 sections and subjects A/B/C/D at 90/70/65/45 produces demand of 180/140/130/90, totalling 540. Each subject fits inside one instructor’s 357, so four instructors suffice, each with spare capacity. If subject C is a coding subject with a single qualified instructor and the college grows to 6 sections, C’s demand becomes 65 x 6 = 390 > 357, so Instructors_C = ceil(390/357) = 2. **That is the exact point at which a second instructor must be hired** — and the system says so, naming the subject and the count.

7.3 The shared ledger — three cases

Case	Rule
Same subject, different batches	An instructor’s calendar is not scoped to a section. If they teach three batches, a free slot must account for commitments to all three. Every placement is checked against, and recorded in, one shared master calendar. The lecture-before-practice chain applies **per batch, independently**.
Same batch, different subjects	One instructor may teach the same section two or more different subjects. This is a normal pattern, not an edge case. A slot booked for one subject blocks the other — same person. Workload must be summed **per instructor**, not per subject; a subject-first workload view will under-count. Each subject keeps its own independent lecture-practice chain, but both compete on the same calendar and must be placed together.

Case	Rule
Ownership across subjects	Ownership is tracked per (subject, section) pair. One instructor can simultaneously own Subject X for Section 1 and Subject Y for Section 1. If they take leave, **every** subject they own for that section must be pulled into resolution — and a single day of leave may require two different substitutes, one per subject.

A recurring lecture-practice pairing failure for one instructor is usually not a scheduling problem — it is a symptom of that instructor being stretched too thin across batches or subjects, and should be read as a capacity issue.

7.4 One owner for the semester

Once an instructor is mapped to a (subject, section), they own it for the entire semester. This is a default owner for every session, not a starting pick.

- A substitute covers only the **specific window of absence** — never a permanent handover.
- The moment the owner returns, they resume full ownership immediately. There is no gradual handback.
- Reassigning a (subject, section) without a leave or exit behind it requires explicit confirmation — it is a larger decision than a routine edit.
- If an instructor permanently leaves, their replacement becomes the new **real owner** going forward, not a standing substitute.

Implementation. Ownership stability is why leave-and-return required a per-session `substitute_instructor` field rather than a change to the ownership model. The owner is unchanged; the sessions inside the absence window carry a stand-in; the validator checks the stand-in's qualification and their own weekly cap. VERIFIED.

8. Disruption handling

The buffer exists for this section. Every mechanism below spends from it, and the Saturday policy chosen at setup is what sized it.

8.1 The absence taxonomy

Nine distinct cases were specified in the system-logic reference. Their current status:

Situation	What is affected	Resolution	Status
Left the organisation	Permanent — every future date, every batch	The template is edited. Sessions become unmapped going forward and the coverage-gap check re-runs.	NOT BUILT
On leave — 1 day	One date, all periods	Each session resolved individually: substitute, reschedule, or bank to recovery.	VERIFIED
On leave — 3+ consecutive days	A date range, all periods	Too many to patch individually — bulk substitute or bulk reschedule, flagging buffer consumption.	VERIFIED
On leave — scattered days	Non-consecutive dates	Resolved as one operation. Repeated collision with the same weekly slot is a structural flag, not a series of one-offs.	PARTIAL
Half-day leave	Back half of one day	Mirror of arriving late.	VERIFIED
Coming in late	First few periods of one day	Try rescheduling later the same day before looking elsewhere.	VERIFIED — via selected-periods
Planned vs emergency leave	—	Planned leave permits fixing in advance; emergency leave can only be resolved after the fact.	PARTIAL
Leave logged after the fact	A date already passed	Treated as a data correction, never as a scheduling problem. The system does not reschedule the past.	NOT BUILT
Leave adjacent to a holiday or weekend	Effective absence exceeds the leave itself	The real gap is counted against the working calendar, since that decides short vs long handling.	NOT BUILT
Returns early	Days already marked missed	Those days are un-cancelled and reactivated; ownership resumes.	NOT BUILT

***Short vs long leave** is decided automatically by the number of affected days (1–2 days patched session-by-session; longer or recurring triggers bulk handling). This threshold must remain configurable.*

Substitute pool exhaustion. If the first-choice substitute is also unavailable, a fallback list applies. If the entire qualified pool for a subject is exhausted simultaneously, that is flagged clearly as "no coverage available" — never left silently unresolved. VERIFIED: a sole-qualified instructor going absent produces recovery items for that subject, and the grid remains valid.

8.2 The unified cascade

> **One rule, applied everywhere**
>
> Any event that removes capacity — faculty absence, a dated holiday, a sudden activity, an exam block — resolves through **one cascade with one set of semantics**. Two paths producing different outcomes for the same lost day is a defect, not a feature. This was enforced after the activity flow and the absence flow were found to disagree.

Step	Mechanism	What changes	Student experience
1	Owner reschedules	A partial-week absence moves the owner’s sessions onto their other days that week.	Same teacher, different day
2	Stand-in substitutes	A full-week gap is covered by a qualified colleague for that week only; the owner returns after.	Same day, different teacher
3	Recovery	No stand-in is free. Affected sessions go to a visible recovery list, never duplicated.	Session deferred, visibly tracked
—	Whole-term reschedule	Every session re-planned with the disruption in force.	**Opt-in only. Never automatic.**

8.3 The conservation rule

The invariant that keeps disruption handling honest. No matter how much shuffling occurs, for every subject:

```
taught_S + still_scheduled_S + buffer_held_S = Required_S
```

Every reallocation moves a slot between "scheduled" and "recovery" — it never destroys one. The subject therefore always ends fully delivered. The recovery check is simply:

```
slots to relocate <= buffer remaining + free slots available
-> if this fails, escalate
```

VERIFIED. Conservation holds on the production term: 2,231 preserved + 34 recovery = 2,265 original. This invariant is what caught a de-duplication defect in which a deleted recovery item could never be re-listed — a genuinely missing session stayed missing while the gap-closing routine reported it fixed.

8.4 Escalation ladder

When the cascade cannot self-heal, escalation is explicit and ordered:

Rung	Action	Status
1	Substitute in place — a qualified instructor free at that slot covers it.	VERIFIED
2	Swap within the section — shift another of that section’s classes in, and move the affected subject to where its owner is free.	PARTIAL
3	Fill now, relocate later — place any subject whose instructor is free, then relocate the affected subject to the earliest valid future slot.	PARTIAL
4	Push to recovery — the slot enters the recovery pool. It still finishes by term end, because buffer was reserved for exactly this.	VERIFIED
5	Escalate — assign a long-term substitute; if still short, convert a holiday to working, add an end-of-day slot, or extend the term.	NOT BUILT — flagged for manual decision

Per the system-logic reference, the system’s role at rung 5 is limited to **surfacing the pacing signal** — "Subject X is 3 sessions behind; on current pace it finishes on [date]" — and letting the coordinator’s chosen fix flow back through the normal calendar logic. It does not solve rung 5 automatically, by design.

8.5 Activities and dated holidays

An activity or holiday announced after generation blocks its cells, orphans the sessions sitting in them, and runs those orphans through the same cascade. The timetable is never wiped and the published version is never overwritten.

Property	Behaviour	Status
Default strategy	manual_recovery — minimum disruption; changed = 0	VERIFIED
Dated holiday on the production term	34 sessions to recovery; 2,231 + 34 = 2,265 conserved	VERIFIED
Whole-term reschedule	Opt-in only; places all 2,265	VERIFIED
Activity granularity	Full day, partial day, selected periods	VERIFIED for absence; activity variants outstanding
Activity scope	University, batch, branch, or selected sections	UNVERIFIED
Cancel	Leaves everything unchanged	UNVERIFIED
Published immutability	A change creates a new draft; published is never overwritten	VERIFIED

Defect found and fixed here. Removing a day renumbers teaching-day indices, so lock pins referenced days that no longer existed. The old code silently fell through to a whole-term reschedule, masking it. Locks are now remapped by **date** to the new indices.

8.6 Impact preview

Before any disruption is applied, the operator must see its cost. Built fields:

Field	Status
Slots consumed	BUILT
Sessions affected	BUILT
Sections affected	BUILT
Faculty affected	BUILT
Remaining buffer	BUILT
Manual attention required	BUILT — via the recovery list
Sessions reassigned	NOT AVAILABLE pre-commit — known only after solving
Sessions rescheduled	NOT AVAILABLE pre-commit — known only after solving
Automatically recoverable	Returns null pre-solve — honestly unknown. See 12.5

9. Exams, versioning, roles, import/export

9.1 Exams — block only

Exam periods reserve dates or periods and prevent regular teaching. The system does **not** generate exam seating, room, or invigilation schedules. This exact wording must appear in the interface adjacent to the exam form.

Critical ordering. Exam, revision and evaluation days must be subtracted **before** the teaching budget is computed, not treated as an activity competing inside it. Leaving them inside the budget would make the capacity view suggest more room for subjects than actually exists.

```
(Working days - exam days - revision days - evaluation days)
x periods per day = actual teaching slot budget
```

Exam period fields: type (Internal, Mid-semester, End-semester, Practical, Supplementary, Custom), start date, end date, applicable university, applicable batches/branches/sections, full-day or selected-period blocking, description.

VERIFIED: 217 slots removed pre-chunk across holidays, exam blocks and activities on the production term. An exam block added after generation requires regeneration; there is no incremental path.

9.2 Versioning — per batch

Each batch maintains its own version history. Different batches may hold different active versions simultaneously. A published version is **never** overwritten — any modification creates a new draft.

```
Draft -> Generated -> Validated -> Pending Approval
-> Published -> Superseded -> Archived
```

Capability	Status
Per-batch independent versions	VERIFIED — batch A at version 0, batch B at version 9, simultaneously
Immutable published versions	VERIFIED — change creates a new draft
Effective-from / effective-until stamping	VERIFIED
Reason for change required	VERIFIED — rejected with 400 if absent
Version comparison and diff	BUILT
Restore an old version as a new draft	BUILT — original preserved
Full version record (approved-by, previous-version, counts, results)	UNVERIFIED
Linked versions across batches for one event	UNVERIFIED
Audit trail with previous and new values	UNVERIFIED
Superseded / Archived states	UNVERIFIED

9.3 Roles and access

Role	Capabilities	Restrictions
Super Admin	All universities; create and manage universities; global rules; manage user access; audit and version history; override or edit any timetable; publish or delegate publication authority.	—
Admin	Assigned universities or the whole organisation per permission; manage calendar, academic structure, faculty and constraints; generate and regenerate; controlled manual edits; override faculty assignment with a reason; all operational reports.	Overrides require a reason, validation, a new version and an audit entry.
BOA	Assigned universities only; configure calendar and academic data; manage batches, sections, subjects and faculty data; upload templates; generate and review; record absences and activities.	Cannot access another university. Cannot manually override sessions. Cannot publish unless separately granted.
Approver / Publisher	Review validated versions; approve or reject; publish; add an approval note; view diffs.	Configurable role — the organisational designation is not hardcoded, pending the business decision on publishing authority.

VERIFIED via the real HTTP path, bypassing the interface: approver cannot generate (403); BOA cannot override a session (403); BOA cannot publish (403); BOA can submit for approval (200); approver can publish (200, transitions to published); a manual edit without a reason is rejected (400). Server-side enforcement is real, not hidden buttons.

9.4 Import

Template-driven, preview-mandatory. The workflow is fixed: **download template** → **fill offline** → **upload** → **preview parsed records** → **review errors** → **correct or confirm** → **save.** Uploaded data is never saved without preview and validation.

Templates required (13): Universities; Batches; Branches and sections; Subjects; Subject-to-section mappings; Faculty; Faculty subject capability; Faculty availability; Holidays; Planned activities; Fixed sessions; Joint sessions; Transfers.

Error taxonomy: missing required columns; duplicate IDs; unknown section; unknown subject; invalid dates; invalid slot counts; faculty reference not found; wrong file format; duplicate transfer records; overlapping transfers.

Status: UNVERIFIED. Without import and clone, entering a real college's data section by section is impractical — this is a pilot blocker in practice even though it is not an engine defect.

9.5 Export

Format	Purpose	Contents
PDF	Official sharing and printing	Section timetable, faculty timetable, batch timetable, version details, effective dates
Excel	Operational review and analysis	Complete timetable, faculty workloads, subject allocation, affected-session reports, version comparison
CSV	System integration and raw exchange	Timetable, recovery list
JSON	Internal service and interface communication	Full result payload with diagnostics

VERIFIED: identical session identities across formats — 210 sessions in CSV equals 210 in JSON on the test instance.

10. User interface specification

10.1 Application areas

```
Dashboard | Setup Wizard | Timetable Workspace
Conflict & Recovery Center | Version History
Import / Export Center | Admin Settings
```

Slim left navigation rail; top context bar carrying current university, batch, academic year, semester, user role and profile menu.

10.2 The setup flow

Step	Screen	Produces
1	Calendar and slot capacity	Semester dates, daily timings, slot duration, day-state policy, multiple breaks, holidays, exam blocks, planned activities — resolved into the per-section slot budget with donut and exact-integer stacked bar.
2	Academic structure	Batches, branches, sections, subjects, credits, exact session counts, multi-section assignment with the separate-vs-joint question, per-section budget and most-constrained indicators.
3	Faculty	Instructor records, capability, weekly capacity, availability, leave, university association — plus a readiness preview. Faculty are never hand-picked per subject.
4	Constraints and events	Fixed sessions with the hard-lock vs soft-preference distinction, optional joint sessions, planned activities, transfer and handover timeline.
5	Review and generate	Readiness checklist categorised Blocking / Warning / Information; pre-generation summary; generation with honest progress stages.

10.3 Readiness gate

The gate blocks only genuine impossibilities and warns on the questionable. Generate remains enabled through warnings. This discipline is verified and is one of the strongest parts of the build.

Condition	Level	Verified
No calendar configured	BLOCK — calendar_missing	Y
Zero teaching slots	BLOCK — no_slots	Y
A subject has no qualified instructor	BLOCK — no_faculty; generation returns INVALID_INPUT	Y
Faculty capacity below demand	BLOCK — capacity_short	Y
No subjects	BLOCK	Y
A section studies no subjects	WARN — generation succeeds; that section receives zero classes	Y

Condition	Level	Verified
An instructor has no subject mapped	WARN — instructor excluded; generation succeeds	Y
A fixed session falls on a non-teaching day	WARN	Y
A subject looks unusually heavy	WARN	Y

Known caveat. *The gate checks capacity as a* ***term total*** *(weekly cap x weeks), while the engine enforces it* ***per week****. A term feasible in aggregate may still be infeasible in a particular week. The gate is therefore permissive — the safe direction, since a gate must never block a configuration that would actually solve — but capacity_short will under-fire. This sentence must appear in the gate's own copy.*

10.4 Timetable workspace

Views: section-wise, faculty-wise, subject-wise, batch-wise, university summary, transfer/handover, activity impact, conflict and recovery.

Grid cell contents: subject, topic/session number, session type, instructor, fixed indicator, joint indicator, transfer phase, validation status. Clicking opens a details drawer.

Controlled editing — no drag-and-drop. Move, swap, change instructor, lock, unlock, cancel, regenerate affected, add sudden activity, record absence, apply transfer, restore version. Admin and Super Admin only. Every action requires a reason, creates a draft version, runs validation, preserves locked and unaffected sessions, and writes an audit record. An invalid edit is never saved.

Drag-and-drop was rejected deliberately: it creates hidden conflicts unless every movement is immediately revalidated, and the faculty-absence auto-rescheduling flow matters more than free-form editing.

10.5 Presentation rules

- **Never colour alone.** Every status carries an icon and a text label. Calendar colours: working day neutral/light blue; half day teal; holiday soft red; exam block amber; activity violet; custom day indigo.
- **Never a donut alone.** Exact integers accompany every proportional chart.
- **Never a fake percentage.** Progress shows named stages, not an invented completion figure.
- **Never a generic failure.** Each of the six result statuses renders distinctly, sourced from real response fields — never inferred.
- **Show the contradiction.** When a configuration is proven infeasible, the contradiction sentence is displayed to the operator. It is the most valuable output the engine produces.
- **Explain assignments from real diagnostics only.** Where the interface explains why an instructor was chosen, every line must come from service output. Invented reasoning is prohibited.

10.6 Visual direction

Modern operational SaaS — closer to Linear or Notion than a student application. Desktop-first. Inter or Manrope. Indigo primary; teal success and working days; amber exams and warnings; violet activities; red reserved for blocking errors only. Soft neutral background, white surfaces, deep charcoal text. 12–16 px card radius, soft shadows, clear table borders, compact data density, subtle gradients on summary cards only, restrained transitions. No neon, no heavy glassmorphism, no decorative animation.

Accessibility: keyboard navigation, visible focus indicators, a label on every field, colour-independent status, sufficient contrast, accessible chart descriptions.

11. Edge-case register

This register consolidates every edge case identified across all three source documents and the verification programme. Status values: **V** verified by an executed run; **P** partial, with a stated limitation; **X** not built or refused; **U** untested; **N/A** deliberately out of scope.

11.1 Calendar and slot capacity

#	Case	Behaviour	St
1	All data valid	Generates. 2,265 sessions, ~4s.	V
2	Zero teaching slots (every day off)	Readiness blocks — no_slots.	V
3	Semester end date before start date	Loud input error naming the field.	V
4	Semester of exactly one day	Handled by the date walk.	U
5	College end time before start time	Loud input error — formerly yielded zero periods silently.	V
6	Slot duration longer than the college day	Loud input error — formerly silent.	V
7	Slot duration leaving an unusable remainder	Remainder discarded; periods fitted to whole slots.	P
8	Each of the 4 Saturday policies	All selectable.	P
9	Half vs full variant for each policy (7 total combinations)	Only 4 policies exposed; the 7-combination matrix is not fully enumerated in the interface.	P
10	Half-day period selection rule	First N periods survive; the remainder are dropped. Fixed rule, deliberately not variable.	P
11	Single break	Periods resume after it.	V
12	Multiple breaks	Periods fitted around every break at real clock times.	V
13	Two breaks overlapping each other	Rejected. Root cause of a prior defect: the validator compared start times, not intervals.	V
14	Break starting before college opens	Rejected.	V
15	Break ending after college closes	Rejected.	V
16	Break exactly at the day boundary	Adjacent-touching is legal by decision; documented.	V
17	Break longer than the college day	Rejected.	V
18	Duplicate break	Rejected.	V
19	Zero or negative break duration	Rejected.	V
20	Holiday outside the semester range	Ignored by the date walk.	U
21	Holiday on a day already off	De-duplicated by the walk — not double-counted.	U
22	Multi-date holiday selection	Supported.	U
23	Date-range holiday selection	Supported.	U
24	Override: a working Saturday forced to holiday	Specified; not exercised.	U
25	Override: an off day forced to working	Required by the recovery mechanism (reclaiming a Sunday). Not exercised.	U
26	Holiday scope — university vs selected batches	Specified; not exercised.	U
27	Every date marked holiday	Readiness blocks — no_slots.	V
28	Exam block overlapping a holiday	Both remove the same slots; idempotent.	U
29	Exam block outside the semester	Ignored.	U
30	Exam block covering the whole semester	Readiness blocks.	U
31	Exam block — full-day vs selected-period	Both supported.	P
32	Activity overlapping an exam block	Both remove slots; idempotent.	U
33	Activity, exam and holiday on the same date	Idempotent.	U
34	Capacity displayed per section, not globally	Per-section throughout.	V

#	Case	Behaviour	St
35	Capacity shown as exact integers, not only a donut	Both required by spec.	P
36	Buffer can go negative (deficit)	Blocks with the exact shortfall named.	V
37	Generalised day state — any weekday full/half/off with cadence	Designed but not built. "Alternate Wednesday off" is inexpressible. See 12.1.	X

11.2 Academic structure and subjects

#	Case	Behaviour	St
38	A section studies no subjects	Readiness warns; generation succeeds; that section receives zero classes.	V
39	A subject mapped to no sections	Ignored — zero demand.	U
40	A subject mapped to every section	Normal; demand scales by section count.	V
41	Duplicate subject code	Rejected at input.	U
42	Subject with zero required sessions	Accepted; contributes nothing.	U
43	Subject with negative required sessions	Rejected at input.	U
44	Subject requiring more sessions than the term contains	PROVEN_INFEASIBLE with the shortfall stated.	V
45	Odd session count on a lecture+practice subject	Rejected at the boundary. This validation formerly could not fire, because the weekly fill silently made odd counts even.	V
46	Credits-to-slots formula visible to the operator	Required by spec.	U
47	Manual slot override	Supported.	P
48	Override without a reason	Must be rejected. Specified.	U
49	One subject to 3 sections — separate delivery	Three independent deliveries.	V
50	One subject to 3 sections — joint delivery	Refused on the production path. See 12.2.	X
51	Batch layer with multiple branches	Modelled; versioning depends on it.	V
52	Branch layer with multiple sections	Modelled.	V
53	Clone subjects from another section	Specified.	U
54	Clone structure from a previous semester	Specified.	U
55	Bulk edit / bulk delete	Specified.	U
56	Topic sequence enforced (topic n before n+1)	Forced violation caught by SEQUENCE. Holds monotonically across all 17 chunk boundaries on the production term.	V
57	Lecture before practice, same day	Forced violations caught by PRACTICE_ORDER and PRACTICE_SAME_DAY. Zero pairs split across a chunk boundary.	V
58	Max sessions per day respected	Enforced via the global cap.	V
59	Per-subject max sessions per day	Not built — a single global cap applies. See 12.4.	X
60	Preferred sessions per week	Removed — was never enforced.	X
61	Consecutive-period requirement	Removed — was never enforced.	X
62	Subject appears at most twice per day	Governed by the day cap.	V
63	Two same-subject sessions must be contiguous	Shares the pairing primitive; not separately exercised.	U

11.3 Faculty and allocation

#	Case	Behaviour	St
64	A subject has no qualified instructor	Readiness blocks; generation returns INVALID_INPUT.	V
65	An instructor with no subject mapped	Readiness warns; instructor excluded; generation succeeds.	V
66	Faculty capacity far below demand	Readiness blocks — capacity_short.	V
67	Capacity exactly equal to demand	Solves with zero slack.	U
68	One instructor qualified for every subject	Allocation balances by least-loaded.	U
69	Two instructors with identical qualifications	Deterministic tie-break.	V
70	Weekly cap enforced	Per calendar week. A crafted 25-against-20 fires INSTRUCTOR_OVER_WEEKLY.	V
71	Daily cap	Not built — column removed.	X
72	Max consecutive slots	Not built — column removed.	X
73	Availability windows honoured	Zero sessions on blocked days.	V
74	Instructor unavailable in every possible slot	PROVEN_INFEASIBLE.	U
75	Manual pin of an instructor to a (section, subject)	Supported.	V
76	Pin contradicting the instructor's qualification	Rejected.	U
77	Pin exceeding the instructor's cap	Rejected or proven infeasible.	U
78	Assignment explanation panel	Must draw on real diagnostics only.	U
79	Instructor workload view	Specified — must be summed per instructor, not per subject.	U
80	Instructor teaching the same section two subjects	Normal pattern. One calendar; both subjects compete on it; workload summed once.	V
81	Instructor teaching the same subject to multiple batches	One shared ledger across all batches.	V
82	Instructor at two universities at the same time	Specified as a readiness error.	U
83	Multi-university association	Modelled.	U

11.4 Transfers, pins and joint sessions

#	Case	Behaviour	St
84	Transfer subsystem on the production term	REFUSED by the chunked solver with a clear message. Formerly a silent drop. See 12.2.	X
85	Transfer with no temporary cover	PROVEN_INFEASIBLE (one-shot).	V*
86	Transfer with no successor	PROVEN_INFEASIBLE (one-shot).	V*
87	Transfer with invalid date order	Rejected (one-shot).	V*
88	Successor equals the departing instructor	Rejected (one-shot).	V*
89	Unqualified successor handing over all deliveries	PROVEN_INFEASIBLE in 0 ms at the post-allocation stage, naming successor and subject. Formerly skipped by an empty loop when the subject list was null.	V*
90	Valid transfer enforced	Incumbent before, successor after (one-shot).	V*
91	Hard pin honoured at its slot	Honoured, including at a chunk boundary.	V
92	Pin on a non-teaching day	INVALID_INPUT, named.	V
93	Two pins on one slot	INVALID_INPUT, named.	V
94	Over-constrained pin	PROVEN_INFEASIBLE.	V

#	Case	Behaviour	St
95	Hard-lock vs soft-preference distinction	Hard causes infeasibility; soft is placed elsewhere and reported.	V
96	Joint session as one synchronised event	Correct one-shot — one topic at one slot for both sections. REFUSED on the production term.	X
97	Joint session where sections have conflicting calendars	Not reachable on the production path.	U
98	Guest session — we provide the slot	Not built.	X
99	Guest session — they provide the slot (highest-priority fixed block)	Not built.	X
100	Visiting faculty availability window	Partial — availability rules exist; no dedicated window model.	P
101	Visiting faculty contract ends mid-semester	Not built — same class as "left the organisation".	X

V* *— verified on the single-shot solver only. The production term runs chunked, which refuses transfers. Their one-shot correctness stands; their production status does not.*

11.5 Disruption

#	Case	Behaviour	St
102	Absent instructor teaches nothing in the window	No-op.	V
103	Absence with a replacement available	Stand-in covers that week; owner resumes.	V
104	Absence with no replacement free	Sessions go to recovery, never duplicated.	V
105	Subject with no qualified cover at all	Refused with a clear message.	V
106	Absence — full day	4 sessions affected; distinguished.	V
107	Absence — half day	Mirror of arriving late.	V
108	Absence — selected periods	2 sessions, period 1 only; distinguished.	V
109	Absence — date range	16 sessions across 3 days.	V
110	Absence spanning two ISO weeks	Substitutions recorded in both chunks.	V
111	Two instructors absent the same week	Two substitutes plus honest recovery; grid valid.	V
112	An instructor and all qualified substitutes absent	Zero substitutes; 50 to recovery; grid valid.	V
113	Sole-qualified instructor absent	Recovery for that subject; grid valid.	V
114	A substitute who is themselves at their weekly cap	The picker enforces the cap and recovers instead.	V
115	Conservation across disruption	2,231 preserved + 34 recovery = 2,265.	V
116	Recovery items never duplicated	Enforced. A de-duplication defect against soft-deleted rows was found and fixed (migration 004).	V
117	Owner resumes after leave (leave-and-return)	Owner unchanged; only in-window sessions carry a substitute.	V
118	Blast radius of a 5-day leave	Reduced from ~300 sessions reassigned to 0 outside the window; the ~20 in-window sessions redistribute.	V
119	Same leave entered before vs after generation	Both paths now produce identical semantics.	V
120	Dated holiday on the production term	Holiday date cleared; 2,265 preserved; default is recovery, not whole-term reschedule.	V
121	Sudden activity — full / partial / selected periods	Absence granularity verified; activity variants outstanding.	U
122	Activity scope — university / batch / branch / sections	Specified.	U

#	Case	Behaviour	St
123	Cancel leaves everything unchanged	Specified.	U
124	Published timetable never overwritten	A change creates a new draft.	V
125	Buffer drained mid-term	Escalation is flagged for a manual decision, not solved automatically.	X
126	Zero-substitute subject, long-term absence	Hard alert — cannot self-heal.	P

11.6 Generation, versioning, roles, import/export

#	Case	Behaviour	St
127	No instructor double-booked	Independently re-validated.	V
128	No section double-booked	Independently re-validated.	V
129	Nobody over their weekly cap	Zero violations on the production grid.	V
130	No unqualified teaching	Applies to substitutes equally.	V
131	Every required session placed	453 per section, exact.	V
132	A deliberately corrupted grid is rejected	Forced double-booking returns validation failure.	V
133	Determinism across runs	Identical grid hash and per-chunk node counts, 3 runs.	V
134	Full term one-shot	UNRESOLVED_TIMEOUT at ~246s, zero placed. Chunking is mandatory, and automatic.	V
135	Bounded search	UNRESOLVED_TIMEOUT reachable in ~1.6s, deterministic.	V
136	Per-batch independent versions	Batch A at 0, batch B at 9, simultaneously.	V
137	Version comparison and restore-as-draft	Original preserved.	P
138	Superseded / Archived states	Specified.	U
139	Linked versions across batches for one event	Specified.	U
140	Audit trail with previous and new values	Specified.	U
141	BOA cannot access another university	Server-side 403 bypassing the interface.	V
142	BOA cannot override a session	Server-side 403.	V
143	BOA cannot publish	Server-side 403.	V
144	Approver cannot generate	Server-side 403.	V
145	Approver can publish	200 — transitions to published.	V
146	Manual edit without a reason	Rejected — 400.	V
147	Export identity parity across formats	210 sessions in CSV equals 210 in JSON.	V
148	Thirteen import templates	Specified.	U
149	Import preview with per-row errors; nothing saved without confirmation	Specified.	U
150	Import error taxonomy (10 classes)	Specified.	U

11.7 The six original open questions

Identified during the initial derivation, before any code existed. Their present status:

#	Question	Resolution	St
151	Topic sequence — must relocation preserve topic order?	Yes. Enforced by construction: the next unplaced topic goes into the next valid slot, so order holds without a post-check. Disruption ripples forward rather than reordering.	V
152	Practice follows its lecture when postponed	Yes. The pair is an atomic unit and moves together. Zero pairs split across chunk boundaries on the production term.	V

#	Question	Resolution	St
153	Section daily capacity — competing lecture+practice pairs	Governed by the per-day cap and the section ledger. A section's day cannot exceed its period count.	V
154	Zero-substitute subjects — sole instructor, long absence	Cannot self-heal. Produces recovery items and a clear "no coverage" signal rather than a silent failure. Automatic escalation is not built.	P
155	Buffer fully drained mid-term — escalation order	Specified as: convert a holiday to working, then add an end-of-day slot, then extend the term. The system flags; the coordinator decides. Not automated.	X
156	Back-to-back same subject — forbid or allow?	Allowed. Twice per day is permitted via the day cap; consecutiveness is unconstrained. The "avoid clustering" preference should be judged per subject, not against an instructor's whole day.	P

12. Known limitations

Each limitation below is stated with the code path it applies to. A limitation that applies only to a non-production path is materially different from one that affects the real term, and this section distinguishes them.

12.1 Generalised day-state model not implemented

Applies to: all paths. **Severity:** design regression.

The system-logic reference specifies that any weekday may be set to fully working, fully off, or half-day, each with a cadence (every week, or alternating 1st/3rd, 2nd/4th, or custom) — stating explicitly that "Saturday was never actually special." The build implements seven hardcoded Saturday combinations and treats every other weekday as fixed. A rule such as "every alternate Wednesday is a half-day" cannot be expressed. This was never consciously excluded; it is a regression from the design, and it also entails that a weekday alternating between states would require two template variants, which the current model has no place for.

12.2 Transfers and joint sessions unavailable on the production path

Applies to: the chunked solver — i.e. every real term. **Severity:** two locked-decision features not delivered.

Both are correct on the single-shot solver used for small instances. The production term is generated chunked, and the chunked path refuses them. Until recently it **silently dropped** them and returned a successful status; the refusal is now explicit and named. Loud refusal is honest, but it is not delivery.

Costed as viable. Joint: the engine represents a joint delivery as a pseudo-section in its owner map, while the chunked allocation, per-chunk assignment and topic-offset logic all key on real (section, subject) pairs — threading requires excluding joint sections from the fixed-assignment path and passing the joint list per chunk. Contained, but real. Transfers: a transfer is a time-sliced owner change, which is precisely the per-delivery per-chunk owner-override mechanism already built for substitutions. A transfer effective on a week boundary maps to clean whole-chunk ownership; one effective mid-week needs the intra-chunk override. The conflict to resolve is that chunking uses fixed global assignments while a transfer changes ownership over time.

Both touch the allocation, assignment and stitch core of the verified 2,265-session path. They are the highest-priority outstanding work and should be undertaken deliberately, not appended to an existing effort.

12.3 Regulatory minimum floor not implemented

Applies to: all paths. **Severity:** compliance exposure.

The design specifies checking allocation against **two** numbers — the internal target and a university-mandated regulatory floor — and flagging when allocation dips below the floor. It further specifies that meeting the floor at setup is insufficient: disruption can push the **delivered** count below the mandated minimum even when the original allocation was sound, so "delivered vs mandated" requires ongoing tracking. Neither the floor nor the tracking exists. The conservation rule guarantees a subject finishes if buffer permits, but nothing asserts a regulatory minimum.

12.4 Per-subject daily cap not implemented

Applies to: all paths. **Severity:** specification gap.

A single global maximum-sessions-per-day applies to every subject. The specification calls for a per-subject value. The global cap cannot express "this subject at most twice a day, but the two-hour laboratory four times." The dead

per-subject column was removed in migration 003 rather than left collecting values nothing enforced.

12.5 Impact preview cannot show reassignment counts pre-commit

Applies to: all paths. **Severity:** minor — with a known remedy.

Sessions-reassigned and sessions-rescheduled are known only after solving, so the preview honestly returns null rather than an invented estimate. That is the correct behaviour given the current design. **However, the remedy is available:** the term solves in approximately 4 seconds, so the preview can run the cascade as a dry-run and report true numbers rather than declining to answer. This is recommended.

12.6 Instructor daily and consecutive caps not implemented

Applies to: all paths. **Severity:** specification gap.

Only the weekly cap binds. Daily and consecutive-slot limits were specified and their columns existed, but nothing enforced them; they were removed rather than left to imply a protection that did not exist.

12.7 Six of seven soft preferences not implemented

Applies to: all paths. **Severity:** low — non-guaranteed by definition.

Avoid first period, avoid last period, prefer mornings, limit instructor gaps, and explicit workload balancing are not implemented. Soft pins are honoured and now reported when dishonoured; subject spreading is applied at placement. The interface must not present unimplemented preferences as available.

12.8 Readiness gate checks capacity as a term aggregate

Applies to: the readiness gate. **Severity:** low — fails in the safe direction.

The gate computes capacity as weekly cap multiplied by weeks; the engine enforces per week. A term feasible in aggregate may be infeasible in a specific week, so capacity_short will under-fire. A gate that is permissive is preferable to one that blocks solvable configurations, but the caveat must be stated in the gate’s own copy.

12.9 Several absence cases not implemented

Applies to: all paths. **Severity:** moderate.

Not built: permanent departure ("left the organisation"); leave logged after the fact (which must be treated as a data correction, never as a scheduling problem — the system must not attempt to reschedule the past); leave adjacent to a holiday or weekend, where the effective absence exceeds the leave and the real gap against the working calendar decides short-versus-long handling; and early return, where days already marked missed must be un-cancelled and ownership resumed.

12.10 Complete product not production-ready

Applies to: the product as a whole. **Severity:** definitional.

Authentication is seeded local accounts, not enterprise SSO. Persistence is SQLite behind a repository abstraction — pilot persistence, not a production database. Deployment, monitoring, concurrency control, backups and operational support do not exist. This is by design for a controlled pilot and is the reason the complete-product verdict is negative regardless of engine quality.

13. Defect taxonomy — engineering findings

This section is included deliberately. The defects found during verification fall into five families, and the families are more valuable than the individual bugs: they describe how a system can be comprehensively tested, fully green, and still wrong. Every one of these was found by executing a run. None was found by reading code — and in several cases, reading the code produced a confident conclusion that a run then disproved.

13.1 The five families

Family	Description	Why it survives testing
A — The value never arrives	A value is entered, stored, and never reaches the solver.	Nothing downstream checks that it arrived. Tests of the field pass because they test storage, not effect.

Family	Description	Why it survives testing
B — The value arrives and means something else	The value reaches the solver and is enforced as a different constraint from the one its name promises.	Every check agrees — in the wrong unit. The validator confirms the solver's own misinterpretation.
C — The check exists, executes, passes, and is wrong	A validation function runs, returns clean, and is checking the wrong thing.	Coverage metrics show the line executed. A passing check is indistinguishable from a working check.
D — Correct on the tested path, dropped on the production path	A feature is genuinely correct on the code path the tests exercise, and silently discarded on the path real data takes.	**The tests are green, honest, and about nothing.** This family defeats every discipline above it.
E — The system substitutes its own number	The system replaces an operator-entered value with a derived one, then validates against its own derivation.	A closed loop can only ever agree with itself. No check can catch it, because every check is downstream of the substitution.

13.2 The defect register

#	Defect	Fam	Consequence	Found by
1	Session counts re-derived by a weekly fill — entered numbers treated as proportions	E	A 453-session syllabus became 798 (+76%), over-committing the term by 112 sessions beyond the 686 that physically existed. All buffer destroyed by construction, so the deficit warning could never fire.	Printing the per-section table of slots vs required vs scheduled
2	Instructor weekly cap forced to a fixed value, discarding the entered figure	E	The readiness gate computed capacity from the entered value while generation used a different one — two numbers for one constraint.	Field-flow trace
3	Solver configuration silently dropped before reaching the solver	A	The node budget could not reach the search phase, so an impossible instance ground for ~60 seconds instead of returning promptly. Presented as a solver defect; was a wiring defect.	Attempting to make UNRESOLVED_TIMEOUT reachable
4	Weekly cap enforced as a whole-term budget rather than per calendar week	B	**Nine of eleven instructors exceeded their contracted cap in some week** — one at 34 against a limit of 20 — on a grid that passed independent validation with zero violations reported.	Printing the per-week load table
5	Break-overlap validation compared start times, not intervals	C	Overlapping breaks were accepted inconsistently.	Testing the full overlap matrix
6	Transfer qualification check skipped when the subject list was null	C	An unqualified successor passed input validation. The release validator caught it after a full solve, so no invalid grid shipped — but the operator received a grid rejection instead of a named input error.	Driving transfer cases that had never been run
7	Recovery de-duplication counted soft-deleted rows	C	A deleted recovery item could never be re-listed: **a genuinely missing session stayed missing while the gap-closing routine reported it fixed.**	Reconciling a test count — on a premise that was itself mistaken
8	Soft-pin reporting and diagnostics discarded when chunks were stitched	D	Every chunk computed the dishonoured-preference report correctly; the stitch threw it away. A dishonoured preference vanished on the real term.	Re-tracing every field against the production path
9	Transfers and joint sessions silently dropped by the chunked solver	D	Both features passed their tests on the single-shot path. On the real term they were ignored and the run returned success. Now refused explicitly.	Re-tracing every field against the production path
10	Removing a day renumbered teaching-day indices, orphaning lock pins	C	Pins referenced days that no longer existed. The old code silently fell through to a whole-term reschedule, masking the fault.	Unifying the activity and absence cascades
11	Low-frequency subjects front-loaded by the chunk splitter	—	A subject needing six sessions was placed in the first six consecutive weeks and then absent for twelve. Every count reconciled perfectly.	Printing the per-week allocation vector

13.3 The lessons

> **Run it; do not read it**
>
> Every wrong conclusion reached during this programme came from reading code. Every real defect came from executing one. The most consequential example: a code inspection reported "there is no per-week enforcement, so the risk cannot arise as feared" — read as reassurance. It was a disclosure. Printing the per-week load table took two minutes and revealed nine of eleven instructors over their cap.

- **A passing check is not a working check.** Three defects were validation functions that ran, returned clean, and were checking the wrong thing. Coverage cannot distinguish them.
- **Test the path the data takes.** A green test on a code path production never executes is worse than no test, because it reports safety that does not exist.
- **Never let the system substitute its own number.** A value the operator entered must be the value the solver uses and the value the validator checks against. If it cannot be honoured, the system must say so — as a block, a warning, or a stated residual — never by quietly using a different figure.
- **Report residuals by direction, never netted.** Twenty under-delivered sessions and sixty-one surplus are not "forty-one over." Over-delivery wastes capacity; under-delivery breaches the syllabus. They cannot be traded.
- **A correct sum can describe a wrong schedule.** The front-loading defect reconciled perfectly at every level. Counts prove conservation; they say nothing about distribution.
- **Reconcile even when the challenge is noise.** The recovery de-duplication defect — silent data loss with a green status — was found by reconciling a test-count discrepancy that turned out not to exist.

14. Verification status and verdicts

14.1 Guarantees verified on the production term

Guarantee	Evidence
Termination — every call returns one of six statuses; never hangs	UNRESOLVED_TIMEOUT reachable in ~1.6s, deterministic across three runs
Conservation — scheduled + recovery = required	2,231 + 34 = 2,265
Exactness — scheduled equals required, per section	453 = 453 on all five sections; zero under-delivery, zero surplus
Independent validation — an invalid grid is rejected, not displayed	A forced double-booking returns validation failure
Determinism — identical inputs produce identical output	Identical grid hash and per-chunk node counts across three runs
No instructor double-booked; no section double-booked	Re-derived by the independent validator on the assembled term grid
Weekly cap — no instructor exceeds their entered cap in any calendar week	Zero violations; a crafted 25-against-20 fires INSTRUCTOR_OVER_WEEKLY
Substitute integrity — stand-ins are qualified and within their own cap	Verified across five absence scenarios
Over-demand — an impossible configuration is proven, not searched	PROVEN_INFEASIBLE in ~0 ms with the contradiction stated
Sequence and pairing hold across chunk boundaries	Monotonic across all 17 boundaries; zero pairs split
Published versions are never overwritten	A change creates a new draft
Server-side permission enforcement	BOA receives 403 calling a privileged endpoint directly, bypassing the interface
Export identity parity	210 sessions in CSV equals 210 in JSON

14.2 Test coverage

```
33 suites / 2,488 assertions / 0 failures
```

Coverage is honest but uneven. The engine core — capacity, allocation, search, validation, chunking, disruption — is verified against the production dataset. Breadth areas — calendar variants, the import subsystem, the version lifecycle, activity scopes — are specified and largely untested. The register in Section 11 states which is which, case

by case.

14.3 Verdicts

Engine — READY FOR CONTROLLED PILOT

Bounded, deterministic, exact, weekly-capped, automatically chunked, with leave-and-return substitution and a unified disruption cascade. Strong on every path tested at production scale. The infeasibility prover returns explained refusals rather than timeouts, and the independent validator has never permitted an invalid grid to ship. **Scope of the claim:** the chunked path on the production dataset. The single-shot path is not viable at this scale and chunking is applied automatically.

User interface and flows — READY FOR CONTROLLED PILOT, WITH TWO STATED HOLES

Transfers and joint sessions do not work on a real term. They are correct on the single-shot solver and refused on the chunked one. The refusal is explicit rather than silent, which is honest — but it is not delivery, and both are locked decisions. The readiness gate, the generation flow, controlled editing, the absence cascade, versioning and role enforcement are verified. The import subsystem and several calendar and version breadth cases remain untested.

Complete product — NOT PRODUCTION-READY

Enterprise authentication, production deployment, monitoring, multi-user concurrency, backups, security review and operational support are out of scope and untested. Persistence is pilot-grade SQLite. This verdict is definitional and does not depend on engine quality. The product must not be described as production-ready until these exist and have been tested.

14.4 Outstanding work, in priority order

#	Item	Rationale
1	Thread joint sessions through the chunked solver	Locked decision, unavailable on every real term. Costed viable and contained.
2	Thread transfers through the chunked solver	Locked decision, unavailable on every real term. Reuses the per-chunk owner-override mechanism already built for substitutions.
3	Impact preview via dry-run	The term solves in ~4s; the preview can report true numbers instead of declining. See 12.5.
4	Sudden-activity variants	Scopes, overlaps, cancel-unchanged, published-immutable.
5	Generalised day-state model	Closes the largest design regression. See 12.1.
6	Import subsystem	Not an engine defect, but a practical pilot blocker: entering a real college section by section is infeasible without it.
7	Calendar breadth — the full 7-combination matrix, overrides, scopes	Largest untested surface.
8	Version lifecycle and audit trail	Superseded/archived states, linked cross-batch versions, previous-to-new value tracking.
9	Regulatory minimum floor	Compliance exposure. See 12.3.
10	Remaining absence cases	Permanent departure, retroactive leave, holiday-adjacent leave, early return. See 12.9.

Appendix A — Formula quick reference

A.1 Capacity

```
Working Days = Total Days - Sundays - Saturday-policy days
- Other declared holidays

Full-day slots = (Window minutes - Break minutes) / Slot length
e.g. (390 - 40) / 50 = 7
```

```
Half-day slots = floor( (Window / 2) / Slot length )
e.g. floor(390 / 2 / 50) = floor(3.9) = 3

Total Available = [ (Total Days - Sundays - Other Holidays
- Full-off Saturdays - Half Saturdays)
x FullDaySlots ]
+ ( Half Saturdays x HalfDaySlots )

Teaching budget = (Working days - exam days - revision days
- evaluation days) x periods per day
```

A.2 Demand and buffer

```
Demand_S = Required_S x Sections_S

Buffer = Total Available - Sum( Required_S )

Buffer >= 0 -> operational reserve
Buffer < 0 -> DEFICIT: refuse, naming the shortfall.
Never display a negative buffer.
```

A.3 Instructor allocation

```
Cap = Working days x Slots per day

Instructors_S = ceil( Demand_S / Cap )

Global check : Sum(Demand_S) <= Total instructors x Cap

Sections/head_S = floor( Cap / Required_S )
Coverable_S = Qualified_S x Sections per head_S
Unallocated_S = max(0, Sections_S - Coverable_S)
Extra needed_S = ceil( Unallocated_S / Sections per head_S )

Ideal utilisation = Sum(Demand) / (Instructors x Cap)
```

A.4 Conservation and recoverv

```
taught_S + still_scheduled_S + buffer_held_S = Required_S

Recovery feasible when:
slots to relocate <= buffer remaining + free slots
-> otherwise escalate
```

A.5 Legality of a placement

```
A session may occupy (day, period) only if:
section is free at that time, AND
owner is free at that time, AND
owner is qualified for the subject, AND
owner is within their weekly cap for that calendar week, AND
the day is a teaching day with that period surviving, AND
topic n is placed after topic n-1, AND
practice shares its lecture's day and follows it, AND
the subject is within its per-day cap for that section

Multi-batch note: instructors live on one real clock.
overlap(a,b) = start_a < end_b AND start_b < end_a
Two sessions clash if they overlap in real time AND share
an instructor or a section. When batches run different
timetables, the free-check runs on wall-clock intervals,
not on slot indices.
```

A.6 Deterministic tie-break

```
1. Batch seniority
2. Subject with fewest qualified instructors
3. Lower section index
```

End of document. *This PRD reflects the verified state of the build at version 1.0. Every claim marked VERIFIED is backed by an executed run against the production dataset. Claims marked otherwise are not, and the distinction is deliberate.*